

DSA Syllabus for SDE Internships

Chapter 1 — Algorithm Analysis & Complexity

Topics

1. What is an algorithm?
2. Characteristics of a good algorithm
3. Time complexity
4. Space complexity
5. Asymptotic notation
 - Big-O
 - Big- Ω
 - Big- Θ
6. Best, average and worst case
7. Constant, logarithmic, linear, linearithmic, quadratic and exponential complexity
8. Analyzing loops
9. Analyzing nested loops
10. Recursive complexity
11. Amortized analysis — basic idea
12. Time-space tradeoff

Practice Questions

1. Find the complexity of a single loop.
2. Find the complexity of nested loops.
3. Analyze a loop where the variable doubles each iteration.
4. Analyze a loop where the variable is divided by 2.
5. Find the complexity of binary search.
6. Find the complexity of merge sort.
7. Find the complexity of recursive Fibonacci.
8. Compare $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$.
9. Analyze the complexity of common array operations.
10. Analyze the complexity of common data-structure operations.

Interview Questions

- What is Big-O notation?
- Difference between Big-O, Big- Ω and Big- Θ ?
- What is the complexity of binary search?
- Why is binary search $O(\log n)$?
- What is the difference between time and space complexity?
- Can an algorithm have $O(n)$ time and $O(1)$ space?
- What is the time complexity of a nested loop?

- Why isn't every nested loop automatically $O(n^2)$?
 - What is amortized complexity?
-

Chapter 2 — Arrays

Topics

1. Static arrays
2. Dynamic arrays
3. Array memory layout
4. Array traversal
5. Insertion
6. Deletion
7. Searching
8. Updating
9. Reversing an array
10. Rotating an array
11. Prefix sums
12. Difference arrays
13. Subarrays
14. Kadane's algorithm
15. Matrix / 2D arrays

Practice Questions

1. Find maximum and minimum element.
2. Reverse an array.
3. Rotate an array.
4. Remove duplicates from a sorted array.
5. Move zeroes to the end.
6. Find the second-largest element.
7. Find missing number.
8. Find duplicate number.
9. Find maximum subarray sum.
10. Find equilibrium index.
11. Find prefix sums.
12. Find pair with a given sum.
13. Merge two sorted arrays.
14. Merge overlapping intervals.
15. Transpose a matrix.
16. Rotate a matrix 90° .
17. Search in a sorted matrix.

Interview Questions

- Why is array access $O(1)$?
 - Why is insertion in the middle $O(n)$?
 - Array vs linked list?
 - Static array vs dynamic array?
 - What is contiguous memory?
 - What is a subarray?
 - How does Kadane's algorithm work?
 - How would you rotate an array in $O(1)$ extra space?
-

Chapter 3 — Strings

Topics

1. Strings and character arrays
2. String traversal
3. String comparison
4. Character frequency
5. Palindrome
6. Anagrams
7. Substrings
8. Subsequences
9. String manipulation
10. String hashing — basic concept

Practice Questions

1. Reverse a string.
2. Check palindrome.
3. Count character frequencies.
4. Find first non-repeating character.
5. Check whether two strings are anagrams.
6. Remove duplicate characters.
7. Find longest common prefix.
8. Reverse words in a string.
9. Check whether one string is a rotation of another.
10. Find longest substring without repeating characters.

Interview Questions

- String vs character array?
- Why are strings often immutable?

- What is the difference between substring and subsequence?
 - How would you check whether two strings are anagrams efficiently?
 - How can you find the first non-repeating character?
-

Chapter 4 — Linked Lists

Topics

1. Linked list concept
2. Node structure
3. Singly linked list
4. Doubly linked list
5. Circular linked list
6. Traversal
7. Insertion
8. Deletion
9. Searching
10. Reversing
11. Fast and slow pointers
12. Cycle detection
13. Merging linked lists

Practice Questions

1. Implement singly linked list.
2. Insert at beginning.
3. Insert at end.
4. Insert at a given position.
5. Delete a node.
6. Reverse a linked list.
7. Find middle node.
8. Detect cycle.
9. Find cycle starting point.
10. Remove nth node from end.
11. Merge two sorted linked lists.
12. Find intersection of two linked lists.
13. Check whether linked list is palindrome.
14. Reverse linked list in groups of K.

Interview Questions

- Array vs linked list?
- Why is random access $O(n)$ in linked lists?

- Singly vs doubly linked list?
 - How does Floyd's cycle detection algorithm work?
 - How can you find the middle node in $O(n)$ time and $O(1)$ space?
 - How would you reverse a linked list without extra memory?
-

Chapter 5 — Stack

Topics

1. Stack as ADT
2. LIFO
3. Push
4. Pop
5. Peek
6. Array implementation
7. Linked-list implementation
8. Applications of stack
9. Infix expression
10. Prefix expression
11. Postfix expression
12. Expression conversion
13. Expression evaluation
14. Monotonic stack

Practice Questions

1. Implement stack using array.
2. Implement stack using linked list.
3. Implement queue using stacks.
4. Check balanced parentheses.
5. Evaluate postfix expression.
6. Convert infix to postfix.
7. Find next greater element.
8. Find next smaller element.
9. Find previous greater element.
10. Find largest rectangle in histogram.
11. Design a Min Stack.

Interview Questions

- What is LIFO?
- Stack vs queue?
- Applications of stack?

- How do you implement a stack using queues?
 - What is a monotonic stack?
 - Why is a stack useful for balanced parentheses?
 - How can you implement Min Stack in $O(1)$?
-

Chapter 6 — Queue & Priority Queue

Topics

1. Queue as ADT
2. FIFO
3. Enqueue
4. Dequeue
5. Linear queue
6. Circular queue
7. Deque
8. Priority queue
9. Heap-based priority queue
10. Applications of queues

Practice Questions

1. Implement queue using array.
2. Implement circular queue.
3. Implement queue using stacks.
4. Implement stack using queues.
5. Design a deque.
6. Generate binary numbers using queue.
7. Sliding-window maximum.
8. Find top K elements using priority queue.
9. Find Kth largest element.

Interview Questions

- Queue vs stack?
 - Why do we need circular queues?
 - What is a deque?
 - What is a priority queue?
 - How does a priority queue differ from a normal queue?
 - What data structure is generally used to implement a priority queue?
-

Chapter 7 — Recursion & Backtracking

Topics

1. Recursion
2. Base case
3. Recursive case
4. Call stack
5. Recursion vs iteration
6. Tail recursion — basic concept
7. Recursive tree
8. Divide-and-conquer recursion
9. Backtracking
10. Decision trees

Practice Questions

1. Factorial.
2. Fibonacci.
3. Sum of numbers.
4. Reverse a string recursively.
5. Binary search recursively.
6. Tower of Hanoi.
7. Generate subsets.
8. Generate permutations.
9. Generate combinations.
10. Combination Sum.
11. N-Queens.
12. Rat in a Maze.
13. Sudoku Solver.

Interview Questions

- What happens in memory during recursion?
- What is a base case?
- Recursion vs iteration?
- When should recursion be avoided?
- What is backtracking?
- Backtracking vs brute force?
- How does N-Queens work?

Chapter 8 — Searching

Topics

1. Linear/sequential search
2. Binary search
3. Requirements for binary search
4. Iterative binary search
5. Recursive binary search
6. First occurrence
7. Last occurrence
8. Lower bound
9. Upper bound
10. Search in rotated sorted array
11. Binary search on answer

Practice Questions

1. Implement linear search.
2. Implement binary search.
3. Find first occurrence.
4. Find last occurrence.
5. Count occurrences.
6. Find insertion position.
7. Find square root using binary search.
8. Search rotated sorted array.
9. Find minimum in rotated sorted array.
10. Find peak element.
11. Find minimum capacity required for a problem using binary search.

Interview Questions

- Why is binary search $O(\log n)$?
 - When can binary search be used?
 - What is binary search on answer?
 - How do you avoid integer overflow in binary search?
 - How do you search a rotated sorted array?
-

Chapter 9 — Sorting

Topics

1. Sorting fundamentals
2. Stable vs unstable sorting
3. In-place vs out-of-place sorting

4. Bubble sort
5. Selection sort
6. Insertion sort
7. Merge sort
8. Quick sort
9. Heap sort
10. Counting sort
11. Radix sort
12. Shell sort
13. Comparison of sorting algorithms
14. Sorting complexity

Practice Questions

1. Implement bubble sort.
2. Implement selection sort.
3. Implement insertion sort.
4. Implement merge sort.
5. Implement quick sort.
6. Implement heap sort.
7. Sort an array containing 0, 1 and 2.
8. Merge two sorted arrays.
9. Merge overlapping intervals.
10. Find inversion count.
11. Find Kth largest element.

Interview Questions

- Merge sort vs quick sort?
- Why is merge sort $O(n \log n)$?
- What is the worst-case complexity of quicksort?
- What is a stable sorting algorithm?
- Which sorting algorithm would you use for linked lists?
- Why is comparison-based sorting lower-bounded by $O(n \log n)$?
- When would counting sort be preferable?

Chapter 10 — Hashing

Topics

1. Hashing
2. Hash function
3. Hash table

4. Hash collision
5. Collision resolution
 - Chaining
 - Open addressing
6. Load factor
7. HashMap
8. HashSet
9. Frequency maps
10. Hashing-based problem solving

Practice Questions

1. Count frequencies.
2. Find duplicate elements.
3. Find first unique element.
4. Two Sum.
5. Group anagrams.
6. Longest consecutive sequence.
7. Subarray with sum K.
8. Count subarrays with a given sum.
9. Find intersection of arrays.
10. Find pairs with a given difference.

Interview Questions

- What is hashing?
- What makes a good hash function?
- What is a collision?
- How are collisions handled?
- What is load factor?
- HashMap vs HashSet?
- Why is HashMap lookup generally $O(1)$?
- Can HashMap lookup become $O(n)$?

Chapter 11 — Trees & Binary Search Trees

Topics

1. Tree terminology
2. Root
3. Parent/child
4. Leaf
5. Height

6. Depth
7. Level
8. Binary tree
9. Full binary tree
10. Complete binary tree
11. Perfect binary tree
12. Binary Search Tree
13. Tree traversal

- Preorder
- Inorder
- Postorder
- Level order

14. BST search
15. BST insertion
16. BST deletion
17. Tree height
18. Balanced trees
19. AVL tree
20. Lowest Common Ancestor

Practice Questions

1. Implement binary tree.
2. Preorder traversal.
3. Inorder traversal.
4. Postorder traversal.
5. Level-order traversal.
6. Find tree height.
7. Count nodes.
8. Count leaf nodes.
9. Check whether two trees are identical.
10. Check whether a tree is balanced.
11. Find diameter.
12. Validate BST.
13. Find LCA.
14. Find kth smallest element in BST.
15. Serialize and deserialize a binary tree.

Interview Questions

- Tree vs graph?
- Binary tree vs BST?
- What is the advantage of a BST?
- Why can BST search become $O(n)$?

- What is a balanced tree?
 - What is AVL?
 - What are the complexities of BST operations?
 - Why does inorder traversal of a BST produce sorted order?
 - BFS vs DFS for trees?
-

Chapter 12 — Heap

Topics

1. Heap
2. Min heap
3. Max heap
4. Complete binary tree
5. Heap representation using arrays
6. Heapify
7. Insert
8. Delete
9. Build heap
10. Heap sort
11. Priority queue
12. Top-K pattern

Practice Questions

1. Implement min heap.
2. Implement max heap.
3. Heapify an array.
4. Build heap.
5. Kth largest element.
6. Kth smallest element.
7. Top K frequent elements.
8. Merge K sorted arrays.
9. Find K closest elements.
10. Median from data stream.

Interview Questions

- Heap vs BST?
- Why is heap represented using an array?
- Min heap vs max heap?
- Complexity of insertion?
- Complexity of extracting minimum?

- How is a priority queue implemented?
 - What is the Top-K pattern?
-

Chapter 13 — Graphs

Topics

1. Graph terminology
2. Directed graph
3. Undirected graph
4. Weighted graph
5. Unweighted graph
6. Cyclic / acyclic graph
7. Connected graph
8. Adjacency matrix
9. Adjacency list
10. BFS
11. DFS
12. Connected components
13. Cycle detection
14. Topological sorting
15. Shortest path
16. Dijkstra
17. Minimum Spanning Tree
18. Prim
19. Kruskal
20. Union-Find / DSU

Practice Questions

1. Implement adjacency list.
2. BFS traversal.
3. DFS traversal.
4. Count connected components.
5. Detect cycle in undirected graph.
6. Detect cycle in directed graph.
7. Number of islands.
8. Flood fill.
9. Clone graph.
10. Topological sort.
11. Course schedule.
12. Shortest path.
13. Dijkstra.

14. Minimum spanning tree.
15. Kruskal.
16. Prim.
17. Union-Find implementation.

Interview Questions

- BFS vs DFS?
 - When would you use BFS instead of DFS?
 - Adjacency matrix vs adjacency list?
 - How do you detect a cycle?
 - What is topological sorting?
 - Where is topological sorting useful?
 - What is Dijkstra's algorithm?
 - Why doesn't Dijkstra work with negative edge weights?
 - Prim vs Kruskal?
 - What is Union-Find?
 - What is a connected component?
-

Chapter 14 — Greedy Algorithms

Your university syllabus explicitly includes greedy algorithms, so keep this as a dedicated chapter.

Topics

1. Greedy approach
2. Local vs global optimum
3. Greedy-choice property
4. Activity selection
5. Fractional knapsack
6. Job sequencing
7. Minimum spanning tree
8. Dijkstra as a greedy-based algorithm
9. Interval problems

Practice Questions

1. Activity selection.
2. Fractional knapsack.
3. Assign cookies.
4. Minimum number of coins — where applicable.
5. Jump Game.

6. Gas Station.
7. Merge intervals.
8. Non-overlapping intervals.
9. Job scheduling.

Interview Questions

- What is a greedy algorithm?
 - Greedy vs dynamic programming?
 - When does greedy fail?
 - How do you prove a greedy solution is correct?
 - Give examples of greedy algorithms.
-

Chapter 15 — Divide and Conquer

Topics

1. Divide
2. Conquer
3. Combine
4. Recurrence relations
5. Binary search
6. Merge sort
7. Quick sort
8. Divide-and-conquer problem solving

Practice Questions

1. Binary search.
2. Merge sort.
3. Quick sort.
4. Maximum subarray using divide-and-conquer.
5. Count inversions.
6. Find majority element.

Interview Questions

- What is divide and conquer?
- Divide and conquer vs recursion?
- Explain merge sort using divide and conquer.
- How do recurrence relations arise?
- Why is merge sort $O(n \log n)$?

Chapter 16 — Two Pointers

This is an important **interview addition** to your university syllabus.

Topics

1. Two-pointer technique
2. Left/right pointers
3. Fast/slow pointers
4. Sorted-array applications
5. In-place algorithms

Practice Questions

1. Two Sum in sorted array.
2. Reverse string.
3. Valid palindrome.
4. Remove duplicates.
5. Move zeroes.
6. Container With Most Water.
7. 3Sum.
8. Linked-list middle.
9. Linked-list cycle.

Interview Questions

- What is the two-pointer technique?
- When can two pointers reduce $O(n^2)$ to $O(n)$?
- Two pointers vs sliding window?
- Why does two-pointer work particularly well on sorted arrays?

Chapter 17 — Sliding Window

Another major interview topic that isn't emphasized in your university curriculum.

Topics

1. Fixed-size window
2. Variable-size window
3. Window expansion

4. Window contraction
5. Frequency-based window
6. Maximum/minimum window

Practice Questions

1. Maximum sum subarray of size K.
2. Maximum number of vowels in a window.
3. Longest substring without repeating characters.
4. Longest substring with K distinct characters.
5. Minimum window substring.
6. Permutation in string.
7. Maximum in every sliding window.

Interview Questions

- What is sliding window?
 - Fixed vs variable sliding window?
 - When should the window expand?
 - When should it contract?
 - Sliding window vs two pointers?
-

Chapter 18 — Backtracking

Topics

1. Backtracking
2. State-space tree
3. Choices
4. Constraints
5. Undoing a choice
6. Pruning
7. Subsets
8. Permutations
9. Combinations

Practice Questions

1. Generate subsets.
2. Generate permutations.
3. Combination Sum.
4. Letter combinations of phone number.
5. N-Queens.

6. Sudoku.
7. Rat in a Maze.
8. Word Search.

Interview Questions

- What is backtracking?
 - Backtracking vs brute force?
 - How does pruning improve backtracking?
 - Explain the state-space tree.
 - What is the complexity of N-Queens?
-

Chapter 19 — Dynamic Programming

This is one of the most important additions for your SDE preparation.

Topics

1. What is DP?
2. Overlapping subproblems
3. Optimal substructure
4. Memoization
5. Tabulation
6. State definition
7. State transition
8. Base cases
9. 1D DP
10. 2D DP
11. Knapsack
12. Subsequence DP
13. Grid DP

Practice Questions

Start with:

1. Fibonacci.
2. Climbing Stairs.
3. House Robber.
4. Coin Change.
5. 0/1 Knapsack.
6. Subset Sum.
7. Longest Common Subsequence.

8. Longest Increasing Subsequence.
9. Unique Paths.
10. Minimum Path Sum.
11. Edit Distance.
12. Word Break.

Interview Questions

- What is dynamic programming?
 - DP vs recursion?
 - DP vs greedy?
 - Memoization vs tabulation?
 - What are overlapping subproblems?
 - What is optimal substructure?
 - How do you identify a DP problem?
 - How do you define a DP state?
 - How do you derive a recurrence?
-

Chapter 20 — Advanced Problem-Solving Patterns

This chapter is essentially your transition from "I know DSA" to "I can solve interview problems."

Topics

1. Prefix Sum
2. Difference Array
3. Fast/Slow Pointer
4. Monotonic Stack
5. Monotonic Queue
6. Merge Intervals
7. Top-K problems
8. Binary Search on Answer
9. Frequency Map
10. Coordinate compression — basic
11. Bit manipulation
12. XOR tricks

Practice Questions

1. Range Sum Query.

2. Product Except Self.
3. Subarray Sum Equals K.
4. Merge Intervals.
5. Insert Interval.
6. Daily Temperatures.
7. Next Greater Element.
8. Top K Frequent Elements.
9. Kth Largest Element.
10. Single Number using XOR.
11. Missing Number using XOR.
12. Maximum Sliding Window.

Interview Questions

- How do you recognize a sliding-window problem?
 - When should you use prefix sums?
 - What is a monotonic stack?
 - What problems are suitable for heaps?
 - When can XOR be useful?
 - How do you recognize binary search on answer?
-

Chapter 21 — Bit Manipulation

This isn't central to your university syllabus, but it is useful for SDE interviews.

Topics

1. Binary representation
2. AND
3. OR
4. XOR
5. NOT
6. Left shift
7. Right shift
8. Checking odd/even
9. Checking/set/unset bit
10. Counting set bits
11. XOR properties
12. Bit masks

Practice Questions

1. Check whether number is odd.

2. Check whether number is power of 2.
3. Count set bits.
4. Find single number.
5. Find missing number.
6. Find two unique numbers.
7. Generate subsets using bit masks.

Interview Questions

- What is XOR?
 - Why does $x \oplus x = 0$?
 - Why does XOR help find a unique element?
 - How do you check whether a number is a power of two?
 - What is a bit mask?
-

Chapter 22 — DSA Revision & Interview Preparation

This is the final stage.

Topics

1. Pattern recognition
2. Choosing the correct data structure
3. Brute force $\rightarrow$ optimized solution
4. Complexity analysis
5. Edge cases
6. Writing clean solutions
7. Explaining your approach
8. Testing your solution
9. Debugging
10. Timed problem solving